SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 3

Comparative Analysis

COURSE NAME: DEEP LEARNING COURSE CODE: MLA04 SLOT : A

Duration: 90 minutes QUESTIONS: 05 | Marks: 25

Name of the Student	Guggilla Venkatasai
Reg. No	192425388
GitHub Link	**https://github.com/venkatasai-eng/MLA0403-Deep learning**

COURSE OUTCOME COVERED

CO3: Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)

CO Weightage: 25% | Duration: 90 minutes | Assessment Tool Weightage: 40% | Marks: 25

Code of Conduct:

I Guggilla Venkatasai(192425388) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:  **Name of the student:** G.Venkatasai

EVALUATION MATRIX & MARKS DISTRIBUTION

Criteria	**Selection of Studies/Parameters (2)**	**Comparison & Differentiation (2.5)**	**Critical Evaluation (2.5)**	**Synthesis & Insight Generation (2)**	**Clarity & Presentation (1)**	**Total (10)**
Weightage	*20 %*	*25%*	*25%*	*20 %*	*10 %*	***100%***
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Signature of the Faculty: _____ Total Marks : _____

Question 1: CNN vs Fully Connected Neural Network

Scenario *A medical imaging organization wants to classify X-ray images using deep learning. Compare a CNN with a traditional fully connected neural network in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.*

1. Context: Why This Comparison Matters for Medical Imaging

Diagnostic X-rays carry meaning in the spatial arrangement of pixels — how a shadow, edge, or density gradient sits relative to its neighbors. A fully connected neural network (FCNN) ignores that structure: it flattens the image into a 1D vector before the first layer even sees it, discarding the 2D relationships that a radiologist relies on. FCNNs also scale very badly with image size. CNNs were built to solve both problems at once, through local receptive fields, shared weights, and a layered feature hierarchy.

2. Architectural Difference: How the Two Networks Connect Neurons

An FCNN connects every neuron in one layer to every neuron in the next — full, pairwise connectivity. A CNN instead uses sparse, local connectivity: each unit in a convolutional layer only looks at a small spatial patch (its receptive field) of the layer before it.

This has a direct effect on how the data is represented. An FCNN collapses a 3D image tensor of shape (height x width x channels) into a single long vector. A CNN keeps that 3D grid intact all the way through its hidden layers, so height, width, and channel information are never lost.

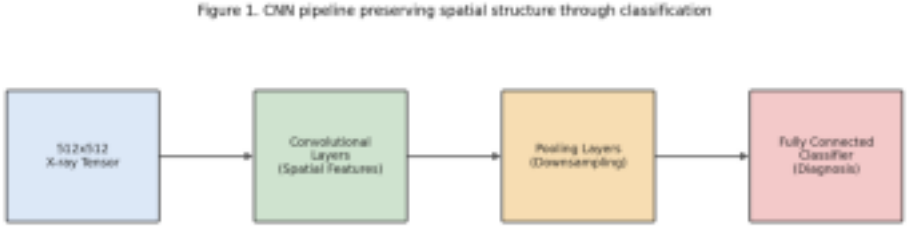

Figure 1. CNN pipeline preserving spatial structure through classification.

3. Parameter and Computational Cost

The scale difference is dramatic. A 512x512 grayscale X-ray has 262,144 pixels. Feeding that directly into an FCNN hidden layer of 1,024 neurons requires (262,144 x 1,024) + 1,024 = 268,436,480 weights — roughly 268 million parameters in just the first layer, which invites overfitting and consumes huge amounts of memory. A convolutional layer with 64 filters of size 3x3x1, by comparison, needs only (3 x 3 x 1 x 64) + 64 = 640 parameters, because the same filters are reused across every position in the image.

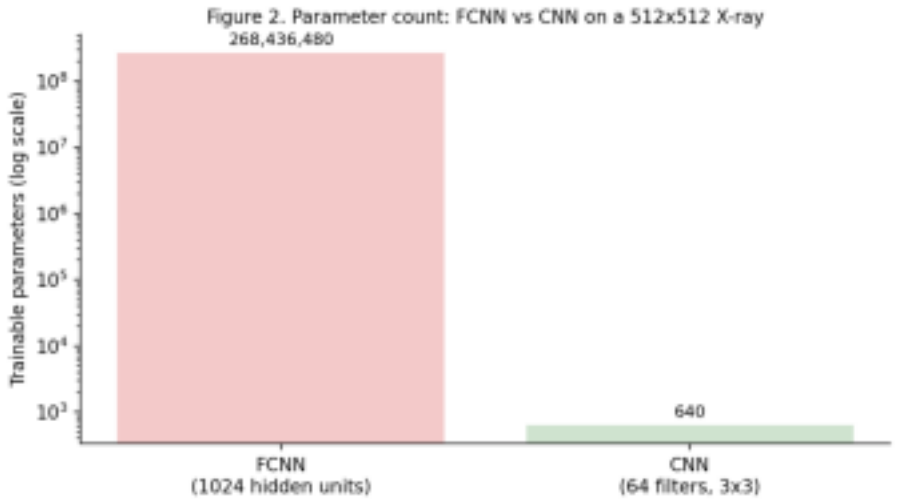

Figure 2. Parameter count: FCNN vs CNN on a 512x512 X-ray (log scale).

4. What Convolution, Filters, and Pooling Actually Do

A convolutional layer performs a sliding cross-correlation: a learnable kernel moves across the spatial grid with a given stride and padding, and at each position it computes a weighted sum of the pixels underneath it, followed by a non-linear activation. The filters themselves act as feature detectors — shallow filters typically pick up tissue boundaries and bone edges, while deeper filters combine those signals into higher-level structures such as rib outlines, lung fields, or abnormal lesions.

Pooling layers (max or average) then shrink the spatial size of the feature maps while keeping the strongest responses, which reduces computation and gives the network some tolerance to small positional shifts.

5. Side-by-Side Summary

- Input representation: FCNN flattens to a 1D vector; CNN keeps a 3D spatial tensor (H x W x C). ● Weight sharing: FCNN learns an independent weight per connection; CNN reuses the same filter across all spatial locations.
- Spatial feature preservation: lost in an FCNN once vectorized; retained throughout a CNN's layers. ● Parameter scale: FCNN grows as input size x hidden size; CNN is bounded by kernel size x channels, independent of image resolution.
- Translation invariance: FCNN is position-sensitive; CNN is robust because a shared filter fires the same way wherever the pattern appears.

6. Why CNNs Are the Better Fit Here

For medical image classification, the case for CNNs is strong. Because they retain spatial orientation, they can pick up a lesion, fracture, or tumor no matter where it sits in the scan. And because they use far fewer parameters, they are less likely to memorize noise in a limited clinical dataset — which translates into more reliable generalization, a property that matters a great deal when the output feeds into a diagnostic workflow.

Question 2: Different CNN Layers and Filters

Scenario: *A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of convolutional layers, pooling layers, activation layers, and fully connected layers in terms of feature extraction, spatial information, computational requirements, and classification. Further compare early-layer filters and deeper-layer filters based on the type of features they learn, and justify how these layers collectively improve defect detection.*

1. Context: Quality Inspection as a Feature-Extraction Problem

Detecting cracks, corrosion, or misalignment on manufactured parts requires picking out faint, localized signals from a cluttered background. A CNN handles this by passing an image through a sequence of specialized layers, each contributing something different to the overall detection task.

2. What Each Layer Type Contributes

- Convolutional layers: the primary feature extractors. They apply local spatial filters and produce multi channel feature maps while keeping the image's coordinate geometry intact.
- Activation layers (e.g., ReLU): inject non-linearity, which is what lets the network learn the complex, non linear boundaries needed to tell a genuine defect apart from a normal surface variation.
- Pooling layers (e.g., max pooling): shrink the spatial grid, which gives some tolerance to where exactly a defect sits and filters out high-frequency noise from surface texture.
- Fully connected layers: take the high-level spatial abstractions built by the earlier layers and map them onto class probabilities — for example, pass versus a specific defect category.

Figure 3. Hierarchical feature extraction for industrial defect detection

Figure 3. Hierarchical feature extraction for industrial defect detection.

3. Comparing the Layers Directly

Figure 4. Qualitative comparison of CNN layer properties

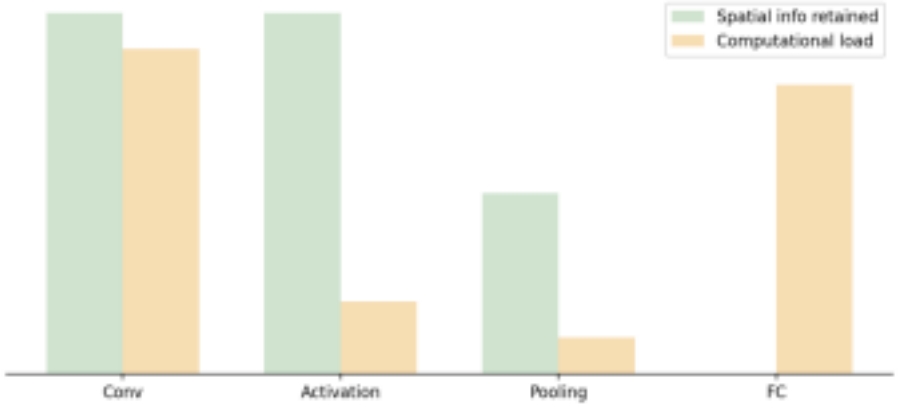

Figure 4. Qualitative comparison of CNN layer properties.

In terms of feature extraction: convolution captures local spatial patterns, activation reshapes them non-linearly, pooling aggregates and downsamples them, and the fully connected layer combines everything into a global, class level decision. Spatial information is fully retained through the convolutional and activation stages, partially reduced by pooling, and completely collapsed once the fully connected layer runs. Computationally, convolution and the fully connected layers are the most demanding, while activation and pooling are comparatively cheap.

4. Early Filters vs Deeper Filters

Filters in the earliest layers have small receptive fields (often 3x3 or 5x5) and pick up low-level primitives: oriented edges, straight lines, and fine texture changes. On a manufactured part, these are exactly the filters that flag hairline cracks and scratches, wherever on the surface they occur.

As the signal passes through more convolution and pooling stages, each unit's effective receptive field grows, and deeper filters start to represent compound, semantic structures — a complete fracture, a missing solder joint, a porosity pattern spread across a wider region. The network essentially builds up from 'what does an edge look like' to 'what does a specific defect type look like.'

5. How the Layers Work Together

The spatial output size of a convolution given input size I, kernel size K, padding P, and stride S follows O = floor((I - K + 2P)/S) + 1. Pooling applies a simple aggregation (e.g., taking the maximum) over a window, and with stride 2 it roughly halves resolution each time — concentrating the strongest feature responses while damping minor artifacts such as mechanical vibration noise in the sensor feed.

Put together, early layers catch localized micro-flaws, activation functions let the network cope with non-linear appearance changes under different factory lighting, pooling keeps detection stable as parts move along the line, and the fully connected layer delivers a fast classification decision — the combination needed for automated, real time quality control.

Question 3: Parameter Sharing vs Independent Weights

Scenario : *A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare parameter sharing in CNNs with independent weights in fully connected networks in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of convolutional filter weights improves learning efficiency and justify its importance for large-scale image processing applications.*

1. Context: Why Efficiency Matters for Surveillance

A city-scale surveillance system ingests a continuous stream of high-resolution frames — often 1080p or 4K — looking for vehicles, pedestrians, and hazards in real time. Processing that volume of data with an FCNN's independent-weight-per-connection design is simply not feasible; parameter sharing in CNNs is what makes real time inference on edge hardware realistic.

2. Independent Weights vs Shared Weights

In an FCNN, every input unit connects to every hidden neuron through its own independent weight — meaning the network effectively has to learn a separate detector for every spatial coordinate in the image. A CNN instead convolves one small set of kernel weights across the entire spatial grid, so the same weight matrix is reused at every location rather than relearned for each one.

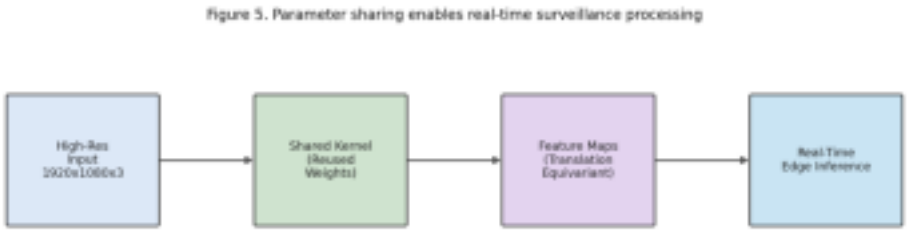

Figure 5. Parameter sharing enables real-time surveillance processing.

3. Quantifying the Difference

Take a 1080p frame of size 1920 x 1080 x 3 — 6,220,800 input values. Connecting that directly to a modest 1,000-neuron FCNN layer requires 6,220,800 x 1,000 ≈ 6.22 billion weights, on the order of 24.8 GB of float32 storage. A convolutional layer with 64 filters of size 5x5, in contrast, needs only (5 x 5 x 3 x 64) + 64 = 4,864 parameters — about 19.4 KB, a reduction factor exceeding a million.

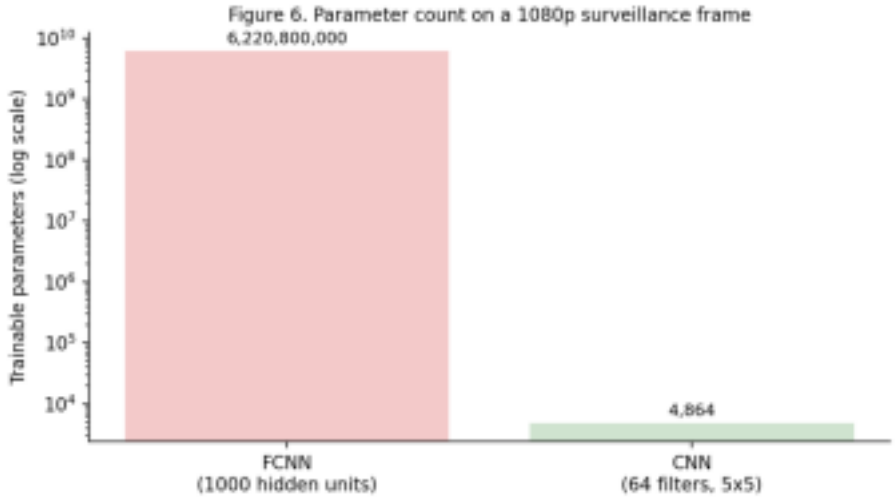

Figure 6. Parameter count on a 1080p surveillance frame (log scale).

4. Comparing the Two Approaches

- Parameter count: FCNN scales as height x width x channels x hidden units; CNN is bounded by kernel size x channels, independent of image resolution.
- Memory footprint: FCNN weights run into gigabytes; CNN weights fit comfortably into megabytes, small enough for an embedded GPU cache.
- Translation property: FCNN is spatially variant — shift an object and it may no longer be recognized; a CNN is translation-equivariant, so a shifted object produces a correspondingly shifted feature response. ● Sample efficiency: an FCNN effectively needs training examples covering every spatial position; a CNN can learn a feature from a few localized instances and apply it everywhere.

5. Why Sharing Improves Learning Efficiency

Parameter sharing works as a built-in regularizer, resting on the assumption of spatial stationarity — that a pattern such as a license plate or a human silhouette looks statistically similar no matter where it appears in the frame. During training, gradients for a shared filter are accumulated across every spatial position where it was applied, so the filter effectively learns from many examples within a single image, not just one. That lets the network generalize from comparatively little training data.

6. Why This Matters for Large-Scale Deployment

For high-resolution, multi-stream video analytics, parameter sharing is what keeps memory demands low enough to run several camera feeds on the same edge device, while preserving translation-equivariant detection across a wide field of view — both essential for a practical surveillance system.

Question 4: AlexNet vs ResNet

Scenario: *A computer vision company is developing an image classification system and is considering AlexNet and ResNet as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.*

1. Context: Two Milestones in CNN Design

AlexNet (Krizhevsky et al., 2012) was the architecture that demonstrated GPU-accelerated deep CNNs could dramatically outperform earlier methods on image classification. ResNet (He et al., 2015) took the field further by introducing residual (skip) connections, which solved the degradation problem that had been blocking networks from scaling past a couple of dozen layers, enabling depths beyond 100 layers.

2. Structural Differences

AlexNet is an 8-layer network — 5 convolutional layers followed by 3 fully connected layers — built around large early filters (11x11, then 5x5), max pooling, and overlapping pooling windows. ResNet, in contrast, is organized as stacks of modular residual blocks (ResNet-50, ResNet-101, ResNet-152, and so on), built from small 3x3 kernels, 1x1 bottleneck convolutions, global average pooling, and identity skip connections.

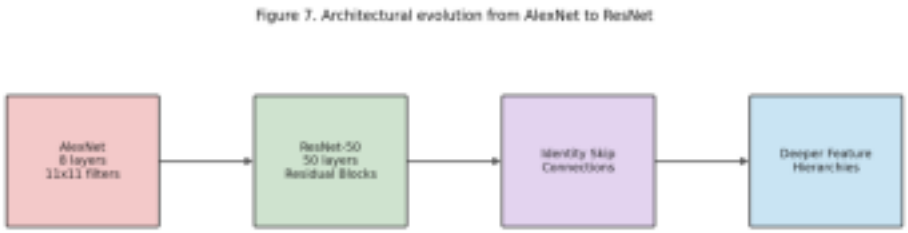

Figure 7. Architectural evolution from AlexNet to ResNet.

3. Depth, Parameters, and Classifier Design

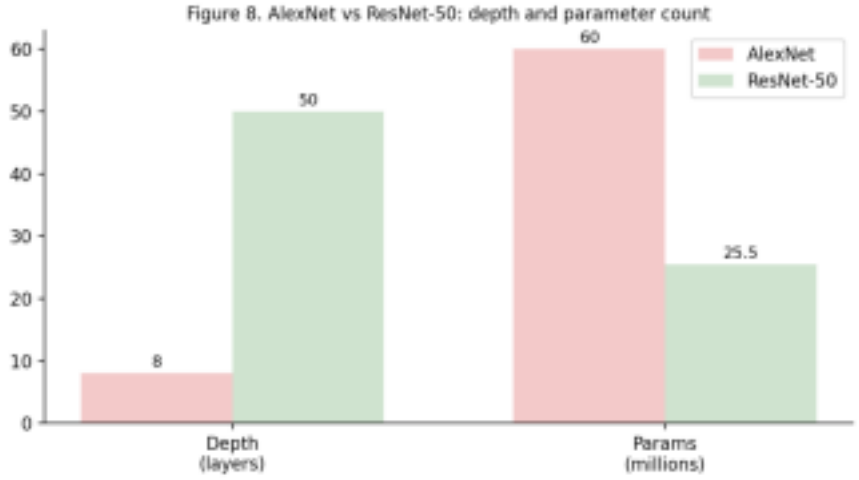

Figure 8. AlexNet vs ResNet-50: depth and parameter count.

AlexNet stays shallow (8 layers) and carries about 60 million parameters, most of them in its two large 4096-unit fully connected layers. ResNet-50 goes far deeper (50 layers) yet uses only about 25.5 million parameters, because global average pooling replaces AlexNet's bulky dense classifier head with a single, much smaller fully connected layer.

4. The Degradation Problem and How Skip Connections Fix It

In a plain sequential network like AlexNet, simply adding more layers eventually hurts training accuracy rather than helping it, largely because gradients shrink as they propagate back through many layers — the vanishing-gradient problem. ResNet's residual blocks reframe what each block learns: instead of learning a full mapping H(x) directly, a block learns a residual F(x) = H(x) - x, and the block's output becomes Y = F(x) + x.

That reformulation changes gradient flow during backpropagation: dL/dx = (dL/dy) x (dF/dx + 1). Even if the learned term dF/dx shrinks toward zero, the constant '1' guarantees a direct path for the gradient back to earlier layers — effectively a shortcut that keeps gradients flowing regardless of depth.

5. Feature Extraction and Efficiency

AlexNet's shallow depth limits it to fairly basic low- and mid-level features, which caps its ability to separate visually similar classes. ResNet's much greater depth lets it build richer, more semantic feature hierarchies, and its global-average-pooling classifier head achieves this with well under half of AlexNet's parameter count — better accuracy at lower memory cost.

6. Recommendation

For a complex image recognition application, ResNet — specifically a variant such as ResNet-50 or ResNet-101 — is the stronger choice. It offers higher classification accuracy, a smaller memory footprint, richer feature extraction, and, thanks to its residual gradient pathways, more reliable and stable training than AlexNet.

Question 5: Regularized CNN vs Non-Regularized CNN

Scenario : *An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a CNN without regularization and a CNN using dropout, data augmentation, and batch normalization in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.*

1. Context: A Classic Overfitting Symptom

Leaf images captured in the field vary in sunlight, background, and orientation. The organization's initial model shows a textbook overfitting pattern — around 99% training accuracy against only about 62% accuracy on unseen field images — which suggests it has memorized incidental details of the training set, such as soil background or a particular lighting condition, rather than learning the actual disease markers.

2. Baseline vs Regularized Architecture

The non-regularized baseline is a standard feedforward CNN trained on raw images with no constraints on how it fits the data, which lets it overfit easily on a limited training set and produces high variance in its predictions. The regularized version adds three complementary mechanisms: dropout (stochastic regularization), data augmentation (expanding the effective diversity of the input), and batch normalization (stabilizing the distribution of intermediate activations).

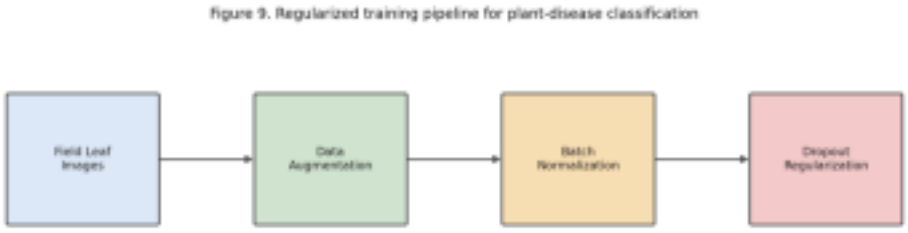

Figure 9. Regularized training pipeline for plant-disease classification.

3. How Each Technique Works

- Data augmentation applies random transformations — rotations, flips, zoom, color jitter — to generate

varied versions of each training image, artificially expanding the diversity the model sees. ● Batch normalization rescales each mini-batch's activations to have stable mean and variance, then applies a learnable scale and shift, which keeps gradients well-behaved and speeds up training.

- Dropout randomly zeroes out a fraction of activations during training, which stops the network from over relying on any single pathway and forces it to learn more redundant, generalizable features.

4. Comparing Outcomes

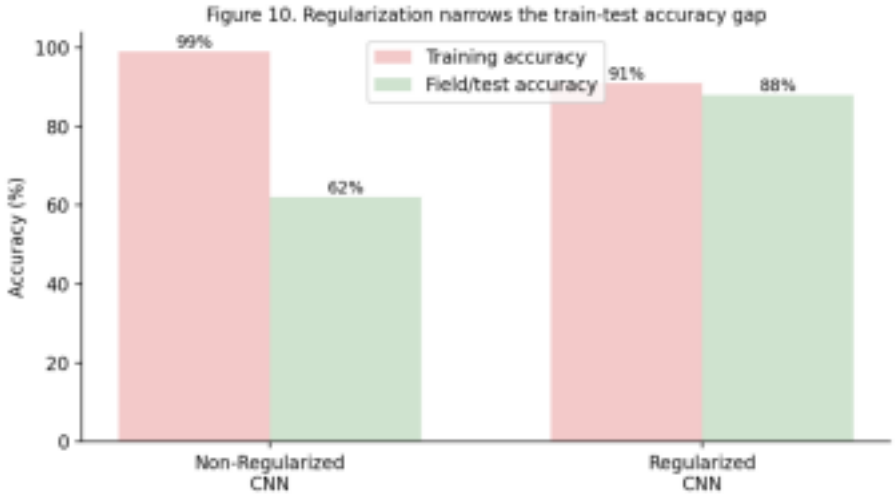

Figure 10. Regularization narrows the train-test accuracy gap.

- Overfitting risk: severe (high variance) without regularization; minimal with it.
- Generalization: poor on unseen field images without regularization; strong across varied field conditions with it.
- Training stability: unstable and sensitive to learning rate without regularization; smooth and stable with batch normalization in place.
- Convergence speed: slower without regularization; faster once batch normalization is added. ● Computational overhead: minimal difference — regularization adds only a modest amount of extra compute during augmentation and normalization.

5. A Practical Implementation Path

A workable rollout looks like this: apply random flips, small rotations (roughly ±25°), brightness adjustments (±20%), and random cropping during training to mimic field variability; insert batch normalization after every convolutional layer and before the activation function; apply dropout (p in the 0.3–0.5 range) inside the fully connected classification layers; and train with an optimizer such as AdamW along with a small L2 weight-decay term (e.g., 0.0001) to keep weights from growing too large.

6. Recommendation

A combined approach — data augmentation, batch normalization, and dropout together — is the right fit for real world plant-disease classification. Augmentation exposes the model to field-like variability, batch normalization keeps training stable, and dropout curbs over-reliance on specific features. Together they close the gap between training and field performance, which is exactly what an outdoor agricultural deployment needs.